

区块链与其他技术

顾辰

guchen@hfut.edu.cn



区块链

- ❑ 区块链的核心思想是建立一个去中心化、不可篡改的分布式账本，使得数据的传输和存储变得更加安全、高效和可靠。
- ❑ 所有的交易都被记录下来，并通过加密技术保护，使得数据不易被篡改或伪造。
- ❑ 不再需要依赖于单一的中心化机构或权威来验证和管理数据，而是由网络中的参与者共同维护。

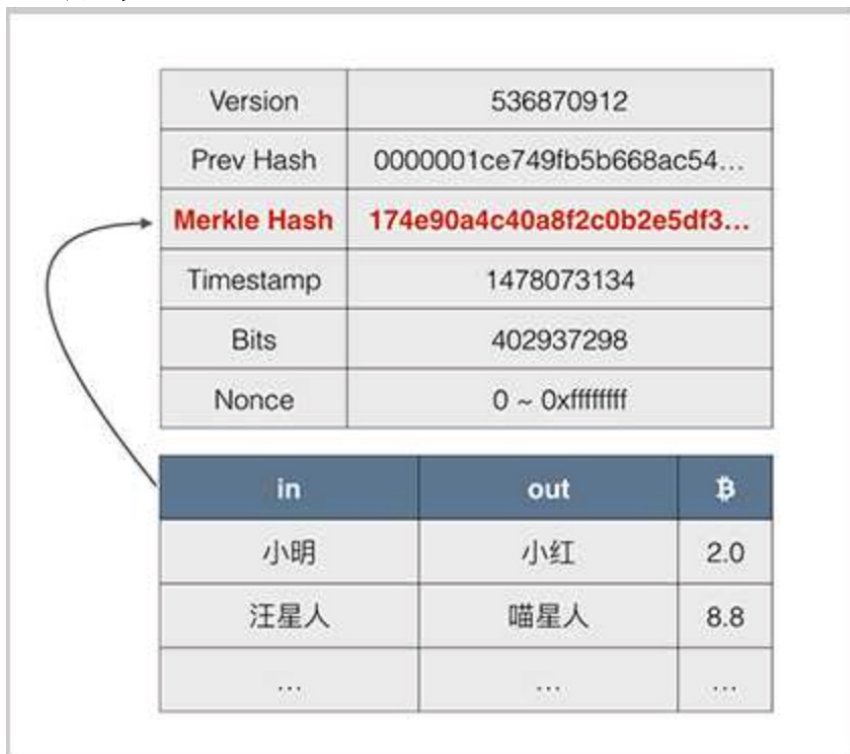


区块链

- ❑ 区块(Block): 区块是区块链中的基本单位, 用于存储数据。每个区块包含了一定数量的交易信息(转账记录等), 还包含区块头的元数据(上一个区块的引用、时间戳、随机数等)
- ❑ 链(Chain): 区块链是由一系列按照特定顺序连接而成的区块构成的链条。每个区块中都包含了指向前一个区块的哈希值, 形成了一个不可篡改的数据链。
- ❑ 分布式账本(Distributed Ledger): 区块链是一种分布式账本, 数据的存储和管理分布在网络中的多个节点上, 而不是集中存储在单一的中心化服务器上。每个节点都包含了完整的账本副本, 并通过共识机制来保持账本的一致性。
- ❑ 去中心化(Decentralization): 区块链的去中心化特性意味着不存在单一的管理者或控制者, 所有的参与者共同维护和管理网络。数据的验证和交易的确认是通过网络中的节点之间的协作和共识达成的, 而不是依赖于单一的中心化机构。

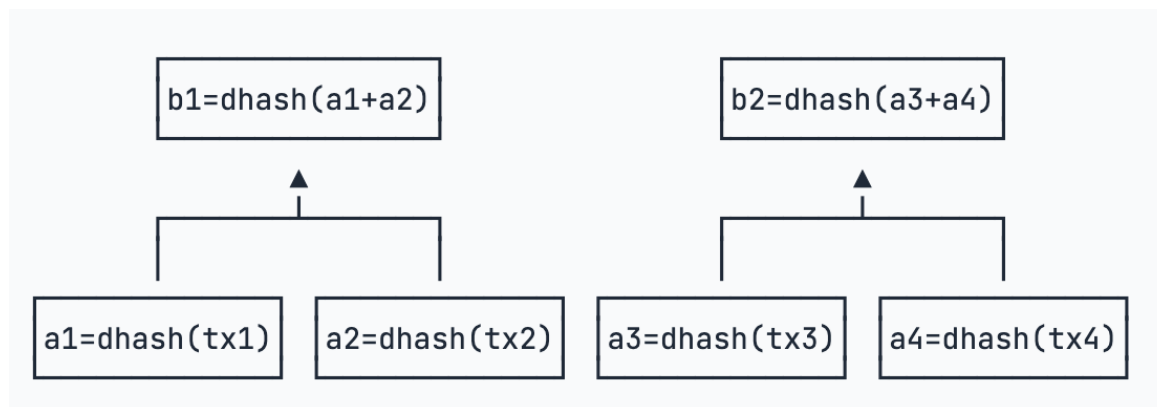
区块链

- 默克尔哈希（Merkle Hash）：在区块的头部，有一个Merkle Hash字段，它记录了本区块所有交易的Merkle Hash。Merkle Hash是把一系列数据的哈希根据一个简单算法变成一个汇总的哈希。



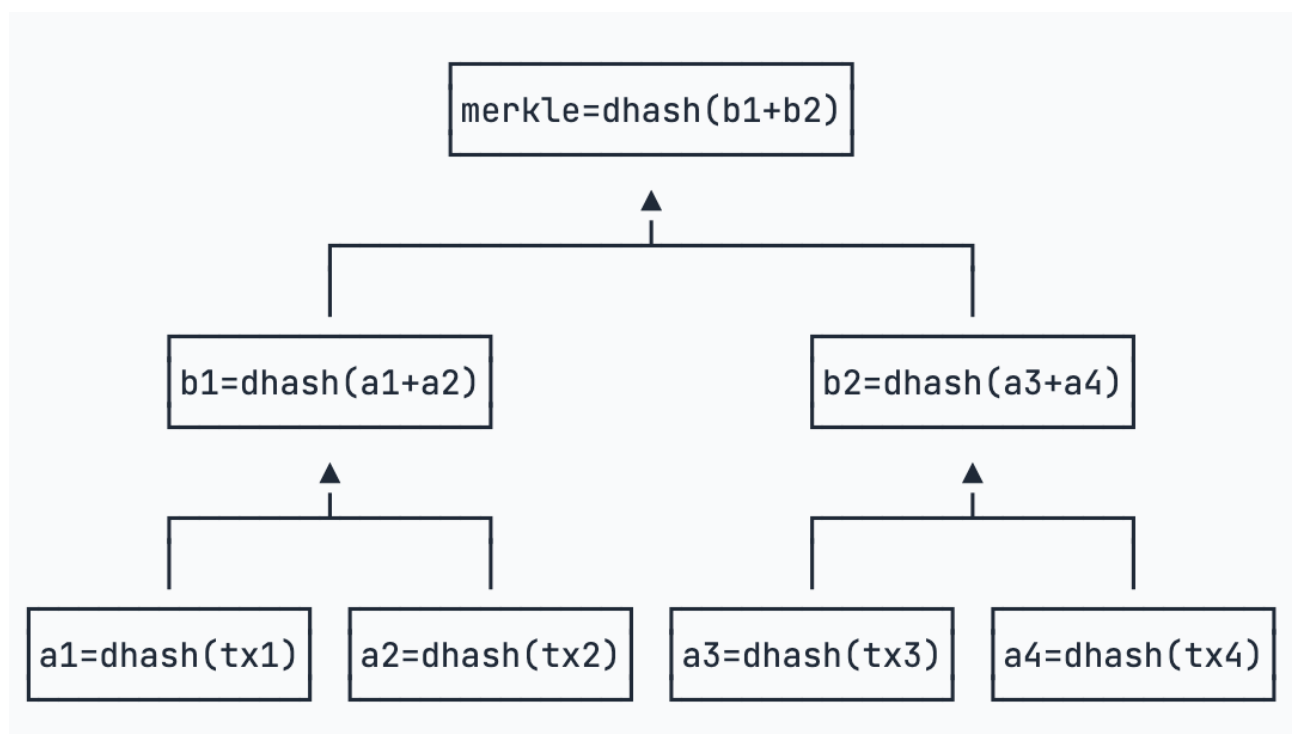
区块链

- 假设一个区块有4个交易，我们对每个交易数据做哈希函数，得到4个哈希值a1, a2, a3和a4。
- 哈希值也可以看做数据，所以可以把a1和a2拼起来，a3和a4拼起来，再计算出两个哈希值b1和b2：



区块链

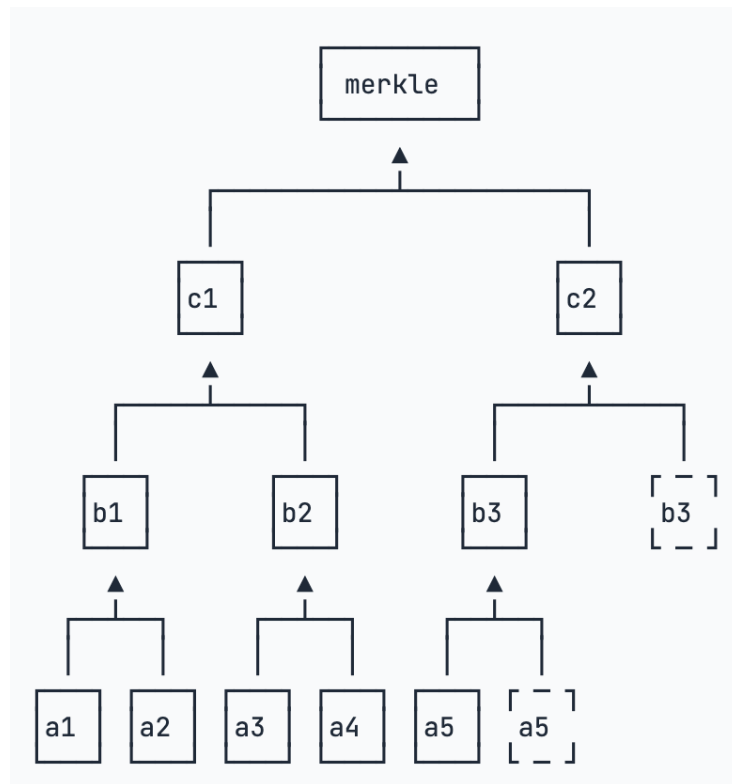
- 最后，把b1和b2这两个哈希值拼起来，计算出最终的哈希值，这个哈希就是Merkle Hash



- 如果交易的数量不恰好是4个怎么办？

区块链

- ❑ 复制已有的哈希值，最后能计算出Merkle Hash。
- ❑ 从Merkle Hash的计算方法可以得出结论：修改任意一个交易哪怕一个字节，或者交换两个交易的顺序，都会导致Merkle Hash验证失败，也就会导致这个区块本身是无效的，所以，Merkle Hash记录在区块头部，它的作用就是保证交易记录永远无法修改。
- ❑ 传统的哈希函数是否可以替代Merkle Hash？



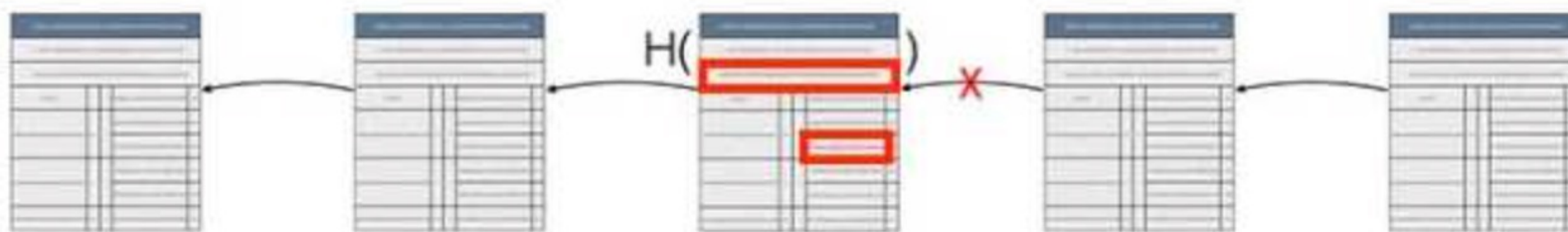
区块链

- ❑ **Block Hash:** 区块哈希标识。一个区块自己的区块哈希并没有记录在区块头部，而是通过计算区块头部的哈希得到。
- ❑ 区块头部的Prev Hash记录了上一个区块的Block Hash，这样，可以通过Prev Hash追踪到上一个区块。
- ❑ 由于下一个区块的Prev Hash又会指向当前区块，这样，每个区块的Prev Hash都指向自己的上一个区块，这些区块串起来就形成了区块链。
- ❑ 区块链的第一个区块（又称创世区块）并没有上一个区块，因此，它的Prev Hash被设置为00000000...000。



区块链

- 如果一个恶意的攻击者修改了一个区块中的某个交易，那么Merkle Hash验证就不会通过。所以，他只能重新计算Merkle Hash，然后把区块头的Merkle Hash也修改了。这时，我们就会发现，这个区块本身的Block Hash就变了，所以，下一个区块指向它的链接就断掉了。
- 由于比特币区块的哈希必须满足一个难度值，因此，攻击者必须先重新计算这个区块的Block Hash，然后，再把后续所有区块全部重新计算并且伪造出来，才能够修改整个区块链。



区块链

- 工作量证明（Proof of Work, PoW）：证明自己做了一定的工作量。例如，在驾校学习了50个小时。而其他人可以简单地验证该工作量。
- 要让计算机实现工作量证明，必须找到一种工作量算法，让计算机无法在短时间内算出来。这种算法就是哈希算法。
- 通过改变区块头部的一个nonce字段的值，计算机可以计算出不同的区块哈希值：

hash (

Version	536870912
Prev Hash	0000001ce749fb5b668ac54...
Merkle Hash	174e90a4c40a8f2c0b2e5df3...
Timestamp	1478073134
Bits	402937298
Nonce	0 - 0xffffffff

)

区块链

- 直到计算出某个特定的哈希值的时候，计算结束。这个哈希和其他的哈希相比，它的特点是前面有好几个0。
- 不断尝试计算区块的哈希，直到计算出一个特定的哈希值。

```
hash256(block data, nonce=0) = 291656f37cdcf493c4bb7b926e46fee5c14f9b76aff28f9d00f5cca0e54  
hash256(block data, nonce=1) = f7b2c15c4de7f482edee9e8db7287a6c5def1c99354108ef33947f34d89  
hash256(block data, nonce=2) = b6eebc5faa4c44d9f5232631f39ddf4211443d819208da110229b644d2a  
hash256(block data, nonce=3) = 00aeaaf01166a93a2217fe01021395b066dd3a81daffcd16626c308c644  
hash256(block data, nonce=4) = 26d33671119c9180594a91a2f1f0eb08bdd0b595e3724050acb68703dc9  
hash256(block data, nonce=5) = 4e8a3dcab619a7ce5c68e8f4abdc49f98de1a71e58f0ce9a0d95e024cce  
hash256(block data, nonce=6) = 185f634d50b17eba93b260a911ba6dbe9427b72f74f8248774930c0d858  
hash256(block data, nonce=7) = 09b19f3d32e3e5771bddc5f0e1ee3c1bac1ba4a85e7b2cc30833a120e41  
...  
hash256(block data, nonce=124709132) = 00000000fba7277ef31c8ecd1f3fef071cf993485fe5eab08e4
```

区块链

- ❑ 假设：计算16次可以在得到的哈希中找到首位是0的哈希值（首位是0出现的概率是1/16）
- ❑ 如果要找出前两位是0的哈希值，理论上需要计算256次（ $16^2=256$ ）；如果要找出前6位是0的哈希值，理论上需要计算 $16^6=1677$ 万次
- ❑ 对于给定难度的SHA-256：假设我们用难度1表示必须算出首位1个0，难度2表示必须算出首位两个0，难度N表示必须算出首位N个0，每增加一个难度，计算量将增加16倍。
- ❑ 对于PoW来说，先给定一个难度值，然后不断变换nonce，计算Block Hash，直到找到一个给定难度值的Block Hash，就算成功。
- ❑ 例如，必须计算出连续17个0开头的哈希值，矿工先确定Prev Hash，Merkle Hash，Timestamp，bits，然后，不断变化nonce来计算哈希，直到找出连续17个0开头的哈希值。17个十六进制的0相当于计算了 16^{17} 次，大约需要计算2.9万亿亿次。



区块链

- 当某个计算方成功找到特定哈希的新区块后，他会立刻向全网广播该区块。其他计算方在收到新区块后，会对新区块进行验证。
- 如果有效，就把它添加到区块链的尾部。同时说明，在本轮工作量证明的竞争中，这个计算方胜出，而其他都失败了。
- 失败的计算方会抛弃自己当前正在计算还没有算完的区块，转而开始计算下一个区块，进行下一轮工作量证明的竞争。
- 为什么区块可以安全广播？因为Merkle Hash锁定了该区块的所有交易，而该区块的第一个coinbase交易输出地址是该矿工地址。每个计算方在挖矿时产生的区块数据都是不同的，所以无法窃取别人的工作量。

比特币

- 比特币是一种虚拟货币，也叫密码学货币。比特币可以脱离物理介质，仅通过互联网就可以流通，而且其手续费低廉，使得小额交易更加方便。
- 全世界所有比特币用户组成的 P2P 网络，比特币是一种**基于 P2P 网络的支付结算系统**。
- 比特币交易是在比特币地址之间完成的，比特币中使用的地址是由公钥的散列值生成的。具体来说，将椭圆曲线 **DSA** 的公钥输入 **SHA-256** 和 **RIPEMD-160** 两个单向散列函数来求出散列值，为其附加一些信息后再通过 **Base58Check** 编码，转换成字符串。

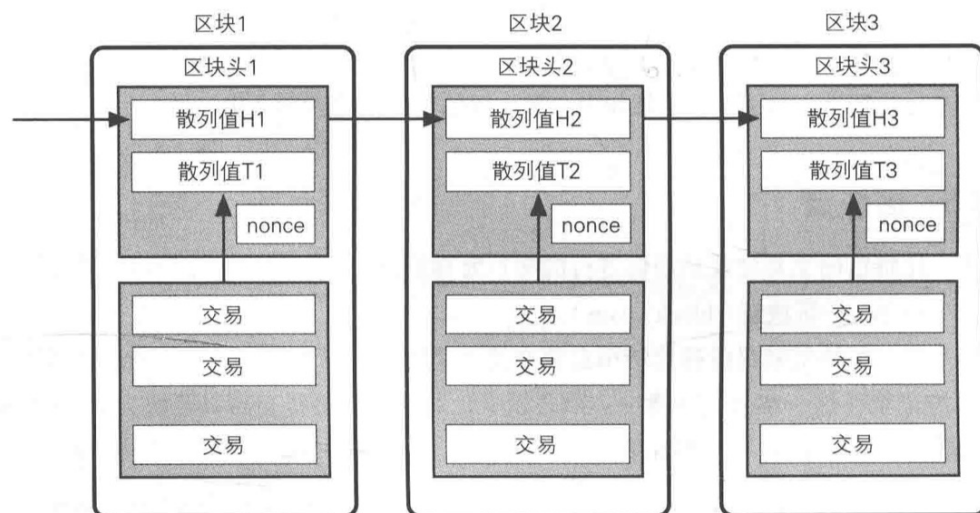
比特币

- 需要使用比特币的客户端，这种客户端称为钱包（wallet）。用户可以在自己的计算机和智能手机上安装钱包应用程序，也可以通过提供钱包服务的网站来使用比特币。用户通过钱包生成密钥对，并据此在互联网上进行交易。其中，公钥用于接收比特币，而私钥用于支付比特币。私钥保存在钱包中，和一般的密钥对的管理方法一样，不能泄露给别人。
- 区块链就是保存比特币全部交易记录的公共账簿。区块链就是将交易以区块为单位组织起来，记录迄今为止所有的交易。



比特币

- 比特币的付款是以交易（**transaction**）为单位来进行的，若干条交易会被合并为一个区块，并被添加到区块链中。当 **P2P** 网络确认区块的添加后，相应的交易也就成立了。
- 一个区块是由若干条交易以及一个区块头所组成的，区块头中保存了上一个区块的区块头的散列值（**H**）。区块头中还保存着本区块所有交易的整体散列值（**T**）。一旦对区块链中的数据进行任何改动，都需要重建所改动的区块之后的所有区块的数据。区块头中的两个散列值有效增加了篡改区块链数据的难度。

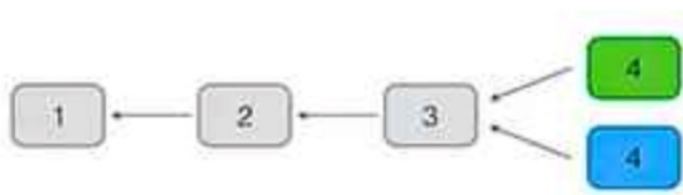


比特币

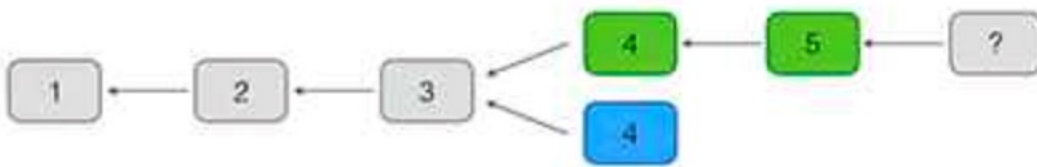
- ❑ 向区块链中添加新的区块就好像从金矿中挖出比特币一样，因此称为挖矿，而从事挖矿的人则称为矿工。
- ❑ 按照比特币协议的规定，成功将区块添加到区块链的矿工将获得挖矿奖励以及该区块所有交易的手续费。
- ❑ 为了防止比特币被伪造，矿工必须证明自己确实完成了规定量的工作，这种证明称为工作量证明。工作量证明是通过散列值来实现的。
- ❑ 比特币网络的难度是不断变化的，它的难度保证大约每10分钟产生一个区块，而难度值在每2015个区块调整一次：如果区块平均生成时间小于10分钟，说明全网算力增加，难度也会增加，如果区块平均生成时间大于10分钟，说明全网算力减少，难度也会减少。因此，难度随着全网算力的增减会动态调整。

比特币

- 通常，矿工接收先收到的有效区块，由于P2P网络广播的顺序是不确定的，不同的矿工先收到的区块是有可能的不同的。这个时候，我们说区块发生了分叉：



- 在分叉的情况下，有的矿工在绿色的分叉上继续挖矿，有的矿工在蓝色的分叉上继续挖矿：



- 但是最终，总有一个分叉首先挖到后续区块，由于比特币网络采用最长分叉的共识算法，绿色分叉胜出，蓝色分叉被废弃，整个网络上的所有矿工又会继续在最长的链上继续挖矿。

权益证明

- 权益证明(Proof of Stake, PoS)与PoW都是随机选择下个区块上传的共识机制。PoW主要是根据算力来决定哪个节点可以进行生产新区块, 算力越大有越大的概率获得打包新区块的机会; 而在 PoS 权益证明中主要就是根据股权, 拥有的股权高即有越高的概率获得生产新区块的机会。
- 在 PoS中, 每一个握有货币的节点都有机会被系统挑选成生产新区块的人, 而这随机挑选的方法是根据权益比重, 意即拥有比较高股权 (货币) 的人即有比较高的概率获得奖励。
- 在 PoS 中每个货币每天都会产生 1 币龄, 若你在权益证明机制中拥有 100 枚货币并存放了 10 天, 你的币龄就为 1,000。若你成功被系统挑选出挖掘新区块, 你的币龄会归 0 并重新开始累积计算, 你会获得的奖励公式如下:
- $\text{奖励} = \text{币龄} * \text{年利率} / 365$
- 假使在一个当前年利率为 5% 的系统中, 你每成功帮忙打包一个新区块会获得的奖励为 $1,000 * 5\% / 365 = 0.137$ 个系统货币。

股份授权证明机制

- 授权 PoS (DPoS, Delegated Proof-of-Stake) 从某种角度来看, DPoS 有点像是议会制度或人民代表大会制度。如果代表不能履行他们的职责(当轮到他们时, 没能生成区块), 他们会被除名, 网络会选出新的超级节点来取代他们。
- 想象有这样一家公司: 公司员工总数有 1000 人, 每个人都持有数额不等的公司股份。每隔一段时间, 员工可以把手里的票投向自己最认可的 10 个人来领导公司, 其中每个员工的票权和他手里持有的股份数成正比。等所有人投完票以后, 得票率最高的 10 个人成为公司的领导。
- 如果有领导能力不胜任或做了不利于公司的事, 员工可以撤销对该领导的投票, 让他的得票率无法进入前 10 名, 从而退出管理层。

可验证延迟函数

- 可验证延迟函数（**Verifiable Delay Function, VDF**）是一类数学函数，能够使得该函数的计算需要至少一段已知的时间，即使是在同时使用少量 CPU 进行并行计算的情况。
- VDF 通常会接受一个输入以及一些参数（安全参数、时间参数等），输出一个结果以及相应的证明（可以为空，如果结果能够自带证明）。验证者会依据输入、参数、输出以及结果来判断 VDF 的结果是否正确。
- VDF 满足以下性质：
 - VDF 的结果的检验应当是非常有效率的。
 - **唯一性**：对于任意一个 VDF 的输入，应当有唯一的输出结果能够通过检验（不存在两个不同的输出，他们有相同的输入）。
 - **串行性**：攻击者即使能够提前计算很长时间（但不是任意长的时间），并且拥有很多并行处理器（但不是任意多的处理器），利用各种计算方法（确定地计算或是连蒙带猜），能够以少于 t 的时间计算出 VDF 结果的概率可以小到忽略不计。

可验证延迟函数

- ❑ VDF 能够抵抗并行计算加速，这意味着为了计算 VDF，应当完成一系列串行才能完成的任务，后一个任务必须依赖于前一个任务。
- ❑ 这时，对哈希函数有所了解的读者可能会想到一种方案：连续将一个输入哈希 t 次。
- ❑ 请问是否可以？
- ❑ 其他的应用：副本证明（**Proof of Replication**）。副本证明所需要解决的问题是，服务器如何向客户端证明自己在某一专门的存储介质上存储了指定的数据，即使这样的数据是可以从别的存储来源轻易获得的。
- ❑ 例如：云存储服务提供商声称自己为客户的数据做了额外的两份冗余备份来保证用户数据的可用性，因此客户需要为这样的冗余备份交更多的钱。但是怎么证明云服务提供商有一共三份副本而不是两份或者只有一份呢？

可编程支付原理

- 比特币的所有交易的信息都被记录在比特币的区块链中，任何用户都可以通过公钥查询到某个交易的输入和输出金额。当某个用户希望花费一个输出时，例如，小明想要把某个公钥地址的输出支付给小红，他就需要使用自己的私钥对这笔交易进行签名，而矿工验证这笔交易的签名是有效的之后，就会把这笔交易打包到区块中，从而使得这笔交易被确认。
- 但比特币的支付实际上并不是直接支付到对方的地址，而是一个脚本，这个脚本的意思是：谁能够提供另外一个脚本，让这两个脚本能顺利执行通过，谁就能花掉这笔钱。
- 比特币交易的输出是一个锁定脚本，而下一个交易的输入是一个解锁脚本。必须提供一个解锁脚本，让锁定脚本正确运行，那么该输入有效，就可以花费该输出。

FROM: UTX0 Hash#index

AMOUNT: 0.5 btc

TO: OP_DUP OP_HASH160 <address> OP_EQUALVERIFY OP_CHECKSIG



可编程支付原理

- 我们以真实的比特币交易为例，某个交易的某个输出脚本是 76a914dc...489c88ac 这样的二进制数据，注意这里的二进制数据是用十六进制表示的，而花费该输出的某个交易的输入脚本是 48304502...14cf740f 这样的二进制数据，也是十六进制表示的：



可编程支付原理

- 我们先来看锁定脚本，锁定脚本的第一个字节76翻译成比特币脚本的字节码就是OP_DUP，a9翻译成比特币脚本的字节码就是OP_HASH160。14表示这是一个20字节的数据（注意十六进制的14换算成十进制是20，于是我们得到20字节的数据）。最后两个字节，88表示字节码OP_EQUALVERIFY，ac表示字节码OP_CHECKSIG，所以整个锁定脚本是：

```
script: 76a914dc5dc65c7e6cc3c404c6dd79d83b22b2fe9f489c88ac
```

```
OP_DUP 76
OP_HASH160 a9
DATA 14 (dc5dc65c...fe9f489c)
OP_EQUALVERIFY 88
OP_CHECKSIG ac
```

可编程支付原理

- 我们再来看解锁脚本。解锁脚本的第一个字节48表示一个72字节长度的数据，因为十六进制的48换算成十进制是72。接下来的字节21表示一个33字节长度的数据。因此，该解锁脚本实际上只有两个数据。

```
script: 4830450221008ecb5ab06e62a67e320880db70ee8a7020503a055d7c45b7
        3dcc41adf01ea9f602203a0d8f4314342636a6a473fc0b4dd4e49b62be28
        8f0a4d5a23a8f488a768fa9b012103dd8763f8c3db6b77bee743ddafd33c
        969a99cde9278deb441b09ad7c14cf740f
```

```
DATA 48 (30450221...68fa9b01)
```

```
DATA 21 (03dd8763...14cf740f)
```

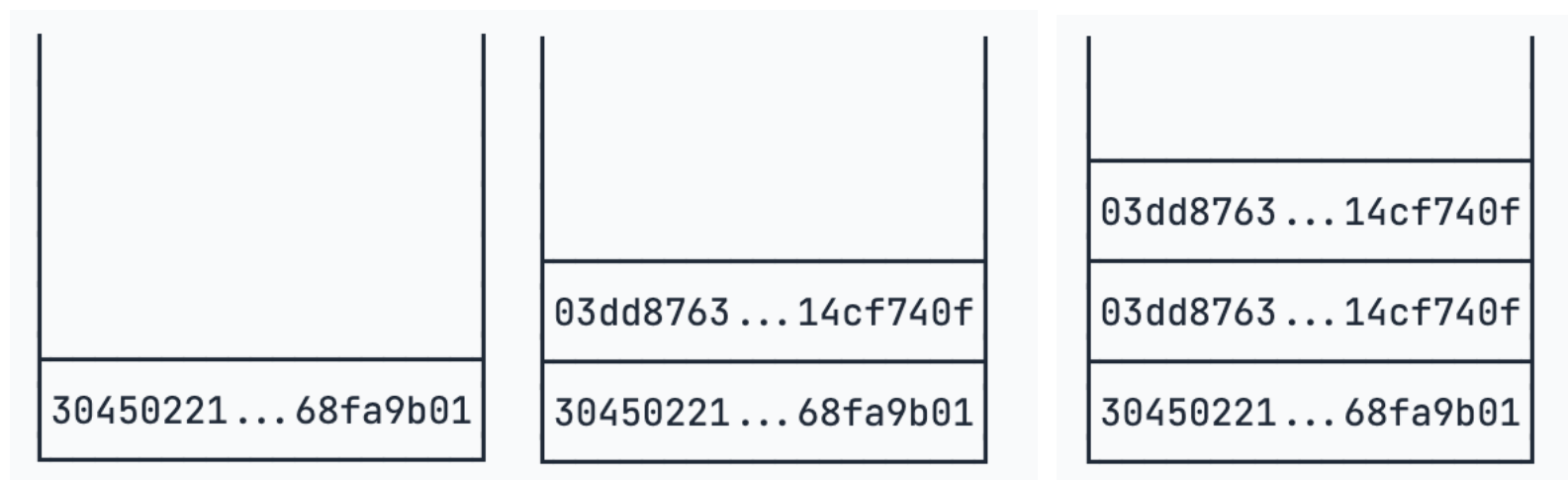
可编程支付原理

- 接下来，我们就需要验证这个交易是否有效。要验证这个交易，首先，我们要把解锁脚本和锁定脚本拼到一块，然后，开始执行这个脚本：

```
DATA 48 (30450221...68fa9b01)
DATA 21 (03dd8763...14cf740f)
OP_DUP 76
OP_HASH160 a9
DATA 14 (dc5dc65c...fe9f489c)
OP_EQUALVERIFY 88
OP_CHECKSIG ac
```

可编程支付原理

- ❑ 比特币脚本是一种基于栈结构的编程语言，所以要先准备一个空栈，用来执行比特币脚本。然后，执行第一行代码，由于第一行代码是数据，所以直接把数据压栈：
- ❑ 紧接着执行第二行代码，第二行代码也是数据，所以直接把数据压栈
- ❑ 接下来执行OP_DUP指令，这条指令会把栈顶的元素复制一份



可编程支付原理

- 然后，执行OP_HASH160指令，这条指令会计算栈顶数据的hash160。对十六进制数据
03dd8763f8c3db6b77bee743ddafd33c969a99cde9278deb441b09ad7c14cf740f计算hash160后得到结果
dc5dc65c7e6cc3c404c6dd79d83b22b2fe9f489c，然后用结果替换栈顶数据：
- 接下来的指令是一条数据，所以直接压栈

dc5dc65c ... fe9f489c
03dd8763 ... 14cf740f
30450221 ... 68fa9b01

dc5dc65c ... fe9f489c
dc5dc65c ... fe9f489c
03dd8763 ... 14cf740f
30450221 ... 68fa9b01

可编程支付原理

- 执行OP_EQUALVERIFY指令，它比较栈顶两份数据是否相同，如果相同，则验证通过，脚本将继续执行，如果不同，则验证失败，整个脚本就执行失败了。
- 最后，执行OP_CHECKSIG指令，它使用栈顶的两份数据，第一份数据被看作公钥，第二份数据被看作签名，这条指令就是用公钥来验证签名是否有效。根据验证结果，成功存入1，失败存入0。当整个脚本执行结束后，检查栈顶元素是否为0，如果不为0，那么整个脚本就执行成功，这笔交易就被验证为有效的。

03dd8763 ... 14cf740f

30450221 ... 68fa9b01

1

可编程支付原理

- 当小明给小红支付一笔比特币时，小明创建了一个锁定脚本，该锁定脚本中引入了小红的地址。要想通过解锁脚本花费该输出，只有持有对应私钥的小红才能创建正确的解锁脚本（因为解锁脚本包含的签名只有小红的私钥才能创建），因此，小红事实上拥有了花费该输出的权利。
- 使用钱包软件创建的交易都是标准的支付脚本，可以编写各种符合条件的锁定脚本。
- 谁能够提供一个数据，它的hash256等于6fe28c0a...，谁就可以花费这笔输出。所以，解锁脚本实际上只需要提供一个正确的数据，就可以花费这笔输出。

OP_HASH256

DATA 6fe28c0ab6f1b372c1a6a246ae63f74f931e8365e15a089c68d6190000000000

OP_EQUAL

可编程支付原理

- 当比特币的脚本通过不同的指令还可以实现更灵活的功能。例如，多重签名可以让一笔交易只有在多数人同意的情况下才能够进行。最常见的多重签名脚本可以提供3个签名，只要任意两个签名被验证成功，这笔交易就可以成功。
- 3个人中，只要任意两个人同意用他们的私钥提供签名，就可以完成交易。这种方式也可以一定程度上防止丢失私钥的风险。3个人中如果只有一个人丢失了私钥，仍然可以保证这笔输出是可以被花费的。

```
FROM: UTX0 Hash#index
```

```
AMOUNT: 10.5 btc
```

```
T0: P2SH: 0P_2 pk1 pk2 pk3 0P_3 0P_CHECKMULTISIG
```

- 从比特币支付的脚本可以看出，比特币支付的本质是由程序触发的数字资产转移。这种支付方式无需信任中介的参与，可以在零信任的基础上完成数字资产的交易，这也是为什么数字货币又被称为可编程的货币。



智能合约

- 由此催生出了智能合约：当一个预先编好的条件被触发时，智能合约可以自动执行相应的程序，自动完成数字资产的转移。保险、贷款等金融活动在将来都可以以智能合约的形式执行。智能合约以程序来替代传统的纸质文件条款，并由计算机强制执行，将具有更低的信任成本和运营成本。
- 智能合约就是一种运行在区块链上的程序。和普通程序不同的是，智能合约要保证在区块链网络的每一个节点中运行的结果完全相同，这样才能使任何一个节点都可以验证挖矿产出节点生成的区块里，智能合约执行的结果对不对。
- 以太坊（Ethereum）是一个支持智能合约的区块链平台，通过一个虚拟机（EVM）可以运行智能合约。以太坊相比比特币的一个重大创新就是它支持智能合约。
- 和普通程序相比，为了消除程序运行的不确定性，智能合约有很多限制，例如，不支持浮点运算（因为浮点数有不同的表示方法，不同架构的CPU运行的浮点计算精度都不同），不支持随机数，不支持从外部读取输入等等。



智能合约

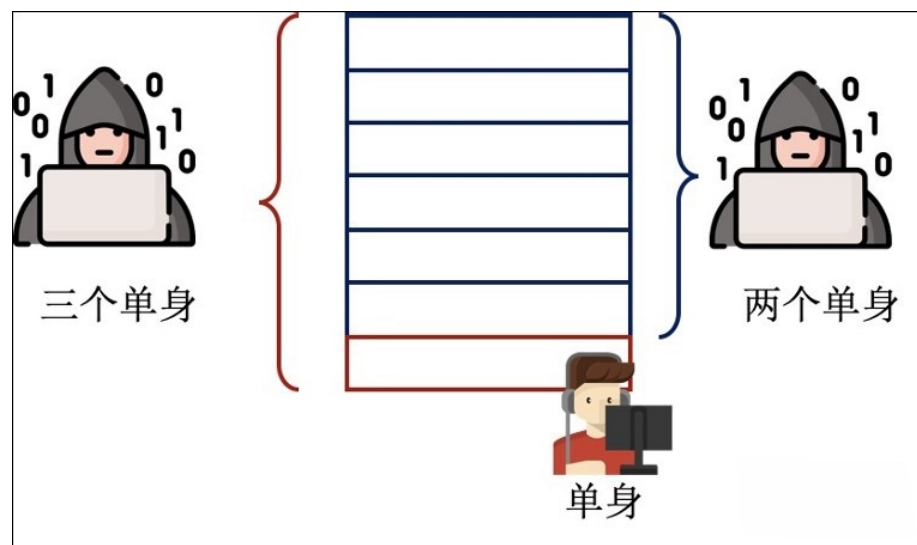
- 类似于Java源码被编译为JVM可执行的字节码，我们也需要一种高级语言来编写智能合约，然后编译成EVM的字节码。最常用的开发智能合约的语言是以太坊专门为其定制的Solidity语言。
- 一个智能合约被编译后就是一段EVM字节码，将它部署在以太坊的区块链时，会根据部署者的地址和该地址的nonce分配一个合约地址，合约地址和账户地址的格式是没有区别的，但合约地址没有私钥，也就没有人能直接操作该地址的合约数据。要调用合约，唯一的方法是调用合约的公共函数。
- 合约在执行的过程中，可以调用其他已部署的合约，前提是知道其他合约的地址和函数签名，这就大大扩展了合约的功能。例如，一个合约可以调用另一个借贷合约的借款方法，再调用交易合约，最后再调用还款方法，实现所谓的“闪电贷”（即在一个合约调用中实现借款-交易-还款）功能。多个合约的嵌套调用也使得因为代码编写的漏洞导致黑客攻击的可能性大大增加。为了避免漏洞，编写合约时需要更加小心。

区块链的局限

- ❑ 区块链技术自诞生以来虽备受关注，但尚未实现大规模普及，核心原因可归结为技术瓶颈、环境制约、生态缺陷等问题。
 - 吞吐量低，延迟高：每秒处理交易远低于传统支付系统。高并发场景下无法满足用户对“实时性”的需求。
 - 存储与计算成本高：区块链的“全节点备份”机制要求每个节点存储完整账本（如比特币账本已超 500GB，以太坊超 1TB），对存储设备和带宽要求极高，普通用户难以承担。
 - 与现有法律体系的冲突：区块链的“不可篡改”、“匿名性”等特性与法律中的“数据更正权”“撤销权”、“实名制”等要求矛盾。
 - 应用场景“伪需求”居多：大量区块链项目为“技术而技术”，未解决传统场景的核心痛点。

差分隐私

- 差分隐私顾名思义就是用来防范差分攻击。假设现在有一个婚恋数据库，2个单身8个已婚，只能查有多少人单身。刚开始的时候查询发现，2个人单身；现在张三跑去登记了自己婚姻状况，再一查，发现3个人单身。所以张三单身。
- 这里张三作为一个样本的出现，使得攻击者获得了额外的知识。而差分隐私需要做到的就是使得攻击者的知识不会因为这些新样本的出现而发生变化。



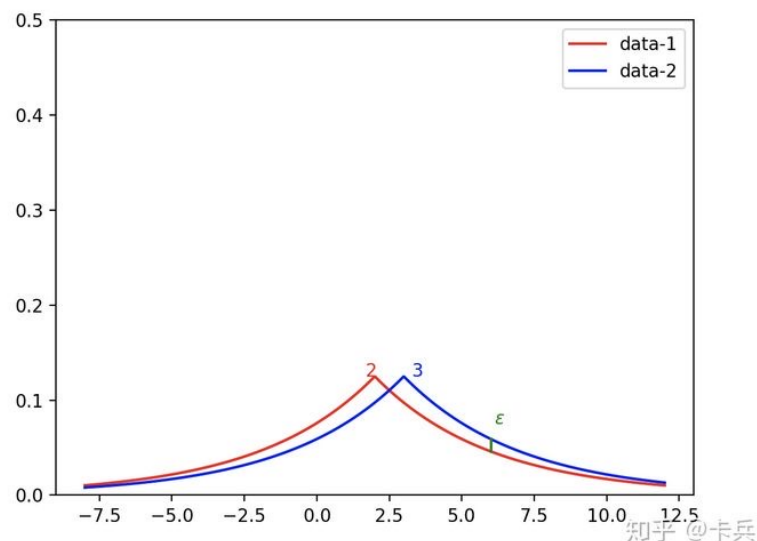
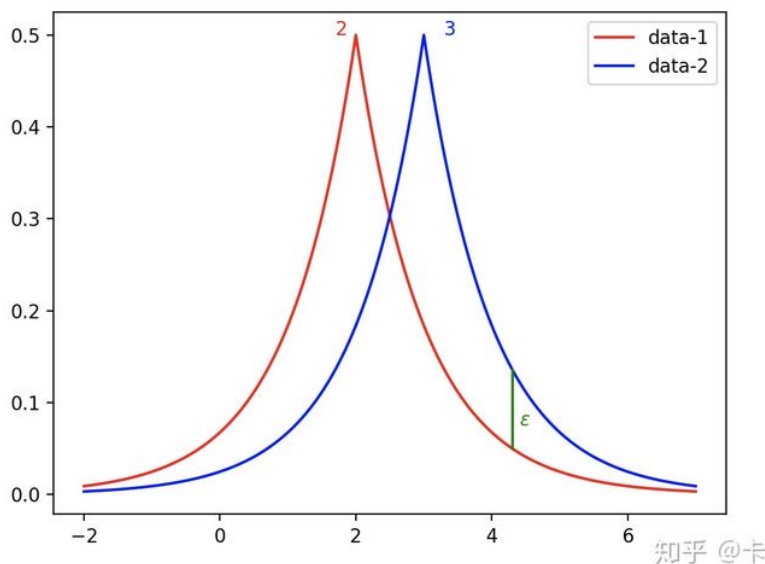
差分隐私的数学定义

查询函数可以用 $f(x) : x \rightarrow R$ 表示（这里暂时只考虑输出结果为1维的情况），随机噪声可以用 r 表示，最终得到的查询结果就是 $\mathcal{M}(x) = f(x) + r$ ，对于两个汉明距离为1的数据集 x, x' ，对于任意的输出集合 S ，应该有：

$$\Pr[\mathcal{M}(x) \in S] \leq e^\epsilon \Pr[\mathcal{M}(x') \in S] \quad (\epsilon \geq 0)$$

差分隐私

- 而 ϵ 就被称为隐私预算，一般而言， ϵ 越小，隐私保护越好，但是加入的噪声就越大，数据可用性就下降了，如下图，数据相对于正确的中心值更加分散。对于应用差分隐私的算法，首先会设定整体的隐私预算，每访问一次数据，就会扣除一些预算，当预算用完，数据就无法再访问。在公式当中 ϵ 是单次查询的预算，而隐私预算一般指的是所有查询所用的预算。



差分隐私实现机制

- 在应用差分隐私进行隐私保护中，需要处理的数据主要分为两大类，一类是数值型的数据，比如说数据集中已婚人士的数量；另外一类是非数值型的数据，比如喜欢人数最多的颜色。这两者，主体分别是数量(连续数据)和颜色(离散数据)。
- 对于数值型的数据，一般采用Laplace或者高斯机制，对得到数值结果加入随机噪声即可实现差分隐私；
- 对于非数值型的数据，一般采用指数机制并引入一个打分函数，对每一种可能的输出都得到一个分数，归一化之后作为查询返回的概率值。

Laplace机制

- 敏感度：对于两个兄弟数据集（只相差一个元素） D 和 D' ，一个查询函数 f 最大的变化范围，比如查询数量，敏感度就是1。敏感度刻画了单个记录改变 f 的输出，最大能改变多少。

$$\Delta f = \max_{D, D'} \|f(D) - f(D')\|_1$$

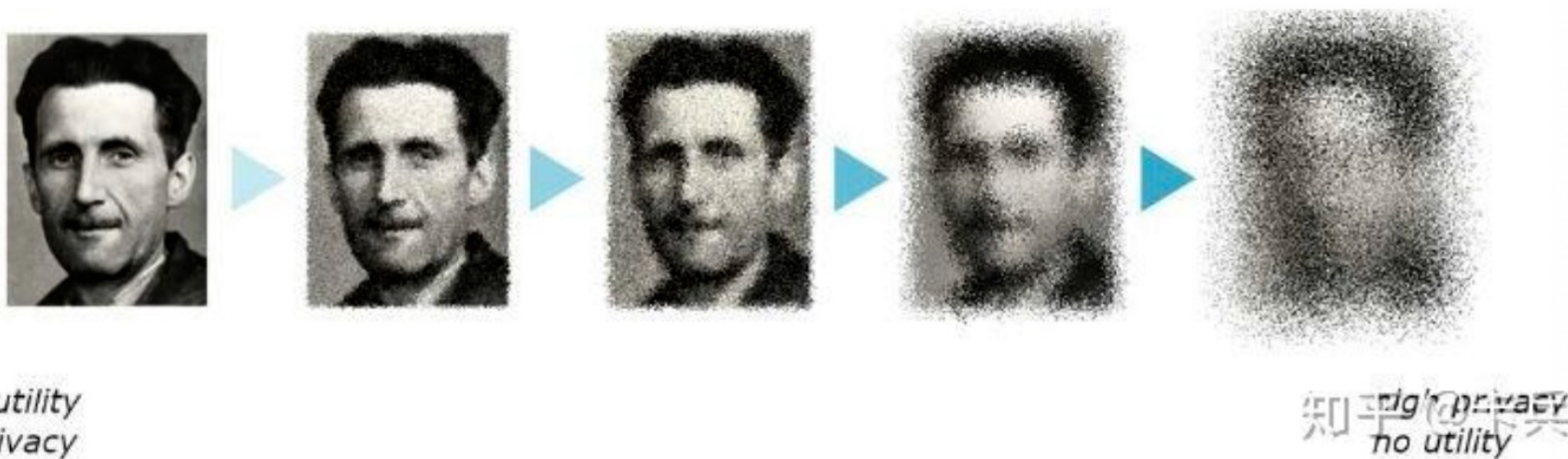
- Laplace机制是在真实的输出加入噪声，噪声在拉普拉斯分布上采样。拉普拉斯分布是一个连续分布，均值为 $\mu=0$ 的拉普拉斯分布的概率密度函数为

$$\text{Lap}(x|b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right)$$

Laplace机制

定义- 噪声 $Y \sim L(0, \frac{\Delta f}{\epsilon})$ 满足 $\epsilon - dp$ 。

其中 Δf 表示敏感度， ϵ 表示隐私预算。可以看到，隐私预算越小，噪声越大，结果可用性越小，隐私保护越好。就像下图一样。隐私预算和可用性成正比。



Laplace机制

- Laplace机制

$$M(D) = f(D) + Y$$

- 其中， $f(D)$ 表示的是查询函数， Y 表示的是Laplace随机噪声， $M(D)$ 表示的是最后的返回结果。
- 拉普拉斯机制是满足 ϵ 差分隐私的。

Laplace机制的证明

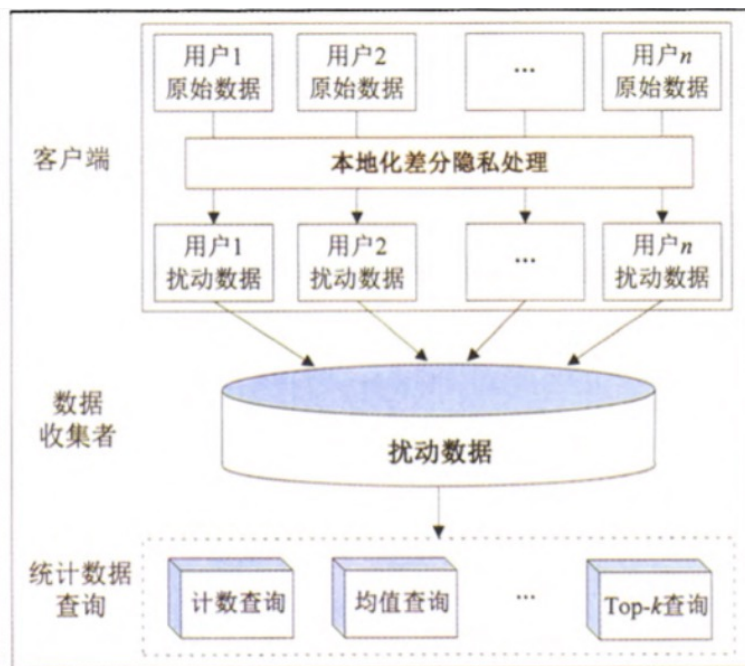
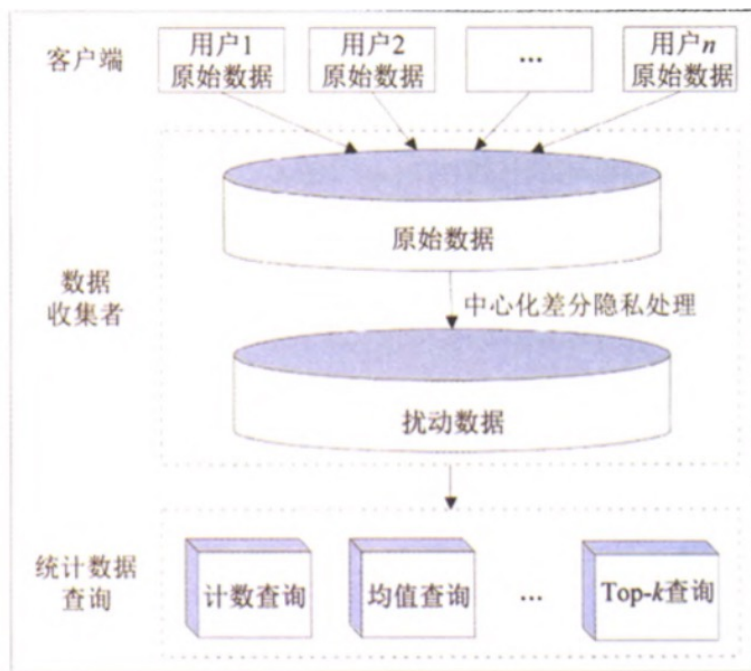
证明：让 $x, y \in \mathbb{N}^{|\mathcal{X}|}$ 满足 $\|x - y\|_1 \leq 1$, $f(\cdot)$ 是 $f: \mathbb{N}^{|\mathcal{X}|} \rightarrow \mathbb{R}^k$. p_x 表示 $\mathcal{M}_L(x, f, \varepsilon)$ 的概率密度函数, p_y 表示 $\mathcal{M}_L(y, f, \varepsilon)$ 的概率密度函数, 我们在任意点 $z \in \mathbb{R}^k$ 比较

$$\begin{aligned}\frac{p_x(z)}{p_y(z)} &= \prod_{i=1}^k \left(\frac{\exp(-\frac{\varepsilon|f(x)_i - z_i|}{\Delta f})}{\exp(-\frac{\varepsilon|f(y)_i - z_i|}{\Delta f})} \right) \\ &= \prod_{i=1}^k \exp\left(\frac{\varepsilon(|f(y)_i - z_i| - |f(x)_i - z_i|)}{\Delta f}\right) \\ &\leq \prod_{i=1}^k \exp\left(\frac{\varepsilon|f(x)_i - f(y)_i|}{\Delta f}\right) \\ &= \exp\left(\frac{\varepsilon \cdot \|f(x) - f(y)\|_1}{\Delta f}\right) \\ &\leq \exp(\varepsilon),\end{aligned}$$

- 其中, 第一个不等式由绝对值不等式得到, 第二个不等式由敏感度 Δf 的定义得到。

中心化差分隐私和本地化差分隐私

- 认为第三方是可信的，因此主要保护的是数据收集分析后的结果发放过程，差分隐私保护机制运行在可信第三方上。
- 认为第三方是不可信的，所以本地差分隐私保护的是用户上传数据到第三方的过程，差分隐私机制运行在各个用户的本地。



零知识证明

- 零知识证明定义：证明者（**prover**）能够在不向验证者（**verifier**）提供任何有用的信息的情况下，使验证者（**verifier**）相信某个论断是正确的。
 - 完备性（**Completeness**）：只要证明者拥有相应的知识，那么就能通过验证者的验证，即证明者有足够大的概率使验证者确信；
 - 可靠性（**Soundness**）：如果证明者没有相应的知识，则无法通过验证者的验证，即证明者欺骗验证者的概率可以忽略。
 - 零知识性（**Zero-Knowledge**）：证明者在交互过程中仅向验证者透露是否拥有相应知识的陈述，不会泄露任何关于知识的额外信息。

交互式零知识证明---色盲游戏

- Alice是色盲，Bob不是色盲，在Bob手上有两个大小形状完全一样的球，但这两个球的颜色不一样，一个球是蓝色的，另一个球是红色的，由于Alice是色盲，所以Alice无法分辨这两个球是否是一样的，Bob需要向Alice证明这两个球是不一样的。在这里，Alice被称为验证者，他需要验证Bob的陈述正确与否，Bob被称为证明者，他需要证明自己的陈述（存在两个颜色不一样的球），Bob需要在Alice不能获得两个球的颜色颜色的情况下，向Alice证明这两个球的颜色是不一样的这个事实，这与零知识证明的定义是相符合的。



交互式零知识证明---色盲游戏

- Alice当Bob的面拿起两个球，左手拿蓝球，右手拿红球，然后将双手放到背后，这样Bob就看不到Alice手上的球了，Alice在背后随机交换左右手上的球，交换完成后Alice将手伸出，并询问Bob两个球是否交换过位置，如果Bob能看到球上的颜色，那么每次Alice换过球的位置后，Bob都能正确回答出Alice的问题。
- 第一次，Alice偷偷交换了手中球的位置，然后Alice问Bob是否交换了球的位置，如果Bob回答Yes，那么Alice有50%的概率相信Bob是可以区分这两个球的颜色，因为Bob有 $1/2$ 的概率蒙对，所以Alice可以在进行一次测试。如果Bob回答No，那么Alice可以肯定Bob不能区分两个球的颜色。
- 第二次，Alice没有交换手中球的位置，然后Alice问Bob是否交换了球的位置。如果Bob回答No，那么Alice有75%的概率相信Bob是可以区分两个球的颜色。

交互式零知识证明---色盲游戏

- 第一次迭代后，Alice可以说Bob陈述的断言为真的概率为50%。如果Bob第二次回答正确，那么Alice可以说Bob陈述为真的概率达75%。在第三次迭代后，它将是87.5%。如果连续n次Bob都通过了检查，则Alice有 $1-(1/2)^n$ 的概率可以认为Bob说的是真的，这两个球的确是有红蓝两种颜色。
- 零知识证明是一种基于概率的验证方式，验证者（verifier）基于一定的随机性向证明者（prover）提出问题，如果证明者都能给出正确回答，则说明证明者大概率拥有他所声称的“知识”。零知识证明并不是数学意义上的证明，因为它存在小概率的误差，欺骗的证明者有可能通过虚假的陈述骗过验证者。换句话说，零知识证明是概率证明而不是确定性证明，但是也存在技术能将误差降低到可以忽略的值。

非交互零知识证明---数独游戏

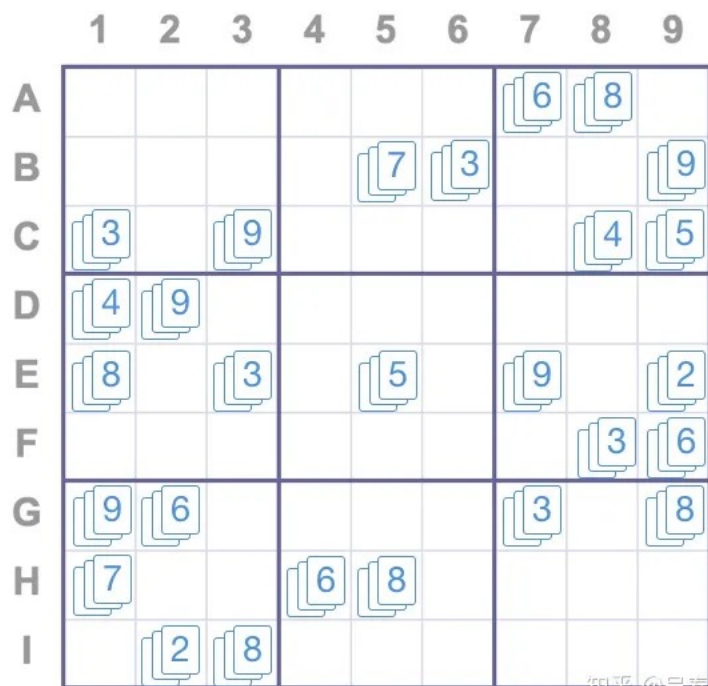
- 交互式零知识证明协议依赖于验证者的随机尝试，需要证明者和验证者进行多次交互才能完成。非交互式零知识证明（Non-Interactive Zero-Knowledge, NIZK）将交互次数减少到一次，可实现离线证明和公开验证。

	1	2	3	4	5	6	7	8	9
A							6	8	
B					7	3			9
C	3		9					4	5
D	4	9							
E	8		3		5		9		2
F								3	6
G	9	6					3		8
H	7			6	8				
I		2	8						

知乎 @吴海鹤

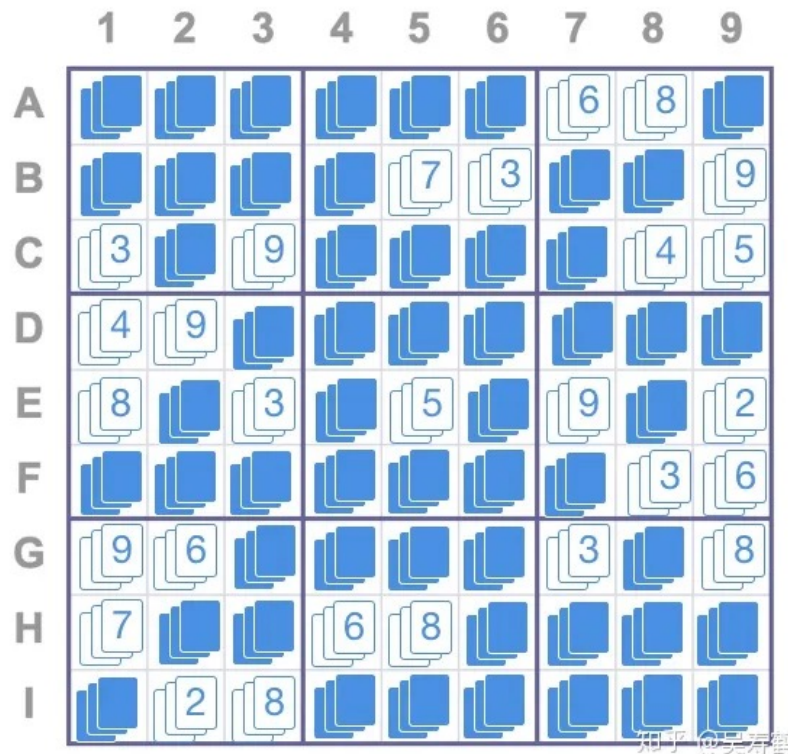
非交互零知识证明---数独游戏

- Alice将生成好的数独答案放入到机器中，机器可以向Bob发送证明。Alice的机器遵循以下公开可验证的协议：
- 首先，Alice在机器中放入尚未被解决的原始数独题目，数独中的谜题卡片三张正面朝上，例如，单元格C3具有3张正面朝上的9号卡片。



非交互零知识证明---数独游戏

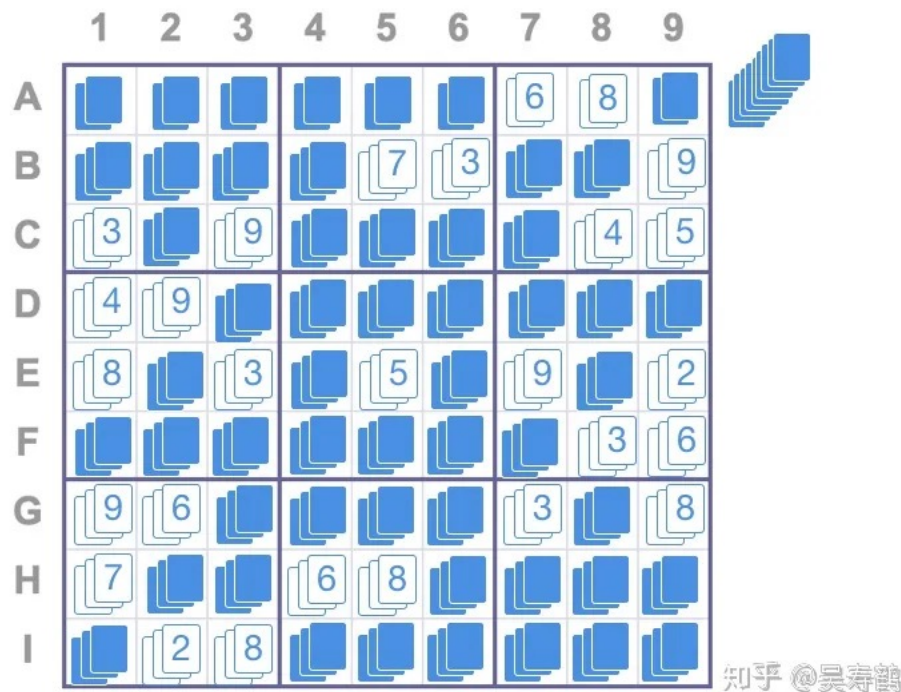
- 接下来，Alice上机器将他的答案正面朝下放置在相应的单元格上，同样也是每个单元放三张。



非交互零知识证明---数独游戏

- 最后Bob向机器获取证明，机器返回给Bob27个袋子：
- 机器将数独中每一行9张卡片取出，并分别混淆后放入一个袋子中，一共有9行，所以9个袋子
- 机器将数独中每一列9张卡片取出，并分别混淆后放入一个袋子中，一共有9列，所以9个袋子
- 机器将数独中每个粗线宫（3*3）内卡片取出，并分别混淆后放入一个袋子中，一共有9个，所以9个袋子

第 52 页



非交互零知识证明---数独游戏

- Bob分别对这27个袋子进行检查，如果每个袋子中的卡片都包含数字1到9，而且没有任何数字丢失或重复，那么Bob可以确认Alice的确解开了数独，并且Bob并没有从机器返回的证明中获取任何关于数独解的知识，因为机器返回给Bob袋子中的数据是被随机打乱的。

零知识证明

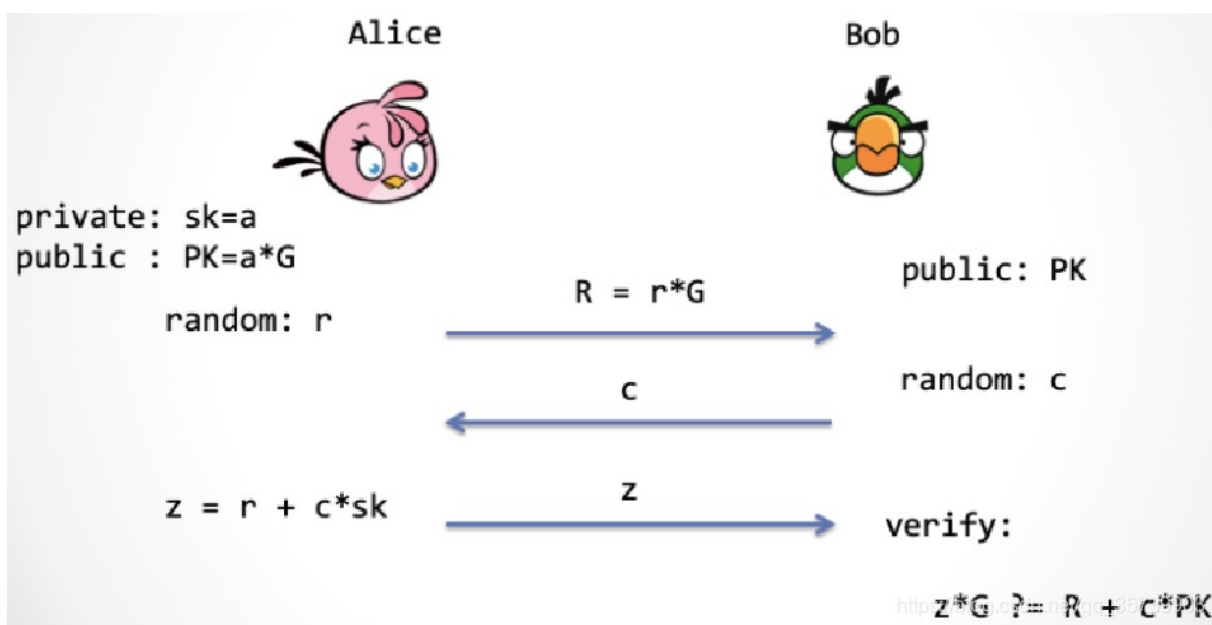
- ❑ 承诺阶段(Commit): 证明者针对命题做出承诺, 并等待验证者提出挑战并进行验证。
- ❑ 挑战阶段(Challenge): 验证者选择随机数, 对提出的承诺进行挑战。
- ❑ 回应挑战阶段(Response): 证明者将收到的随机数并结合给出的承诺, 返回挑战的回应。
- ❑ 验证阶段(Verify): 验证者验证, 挑战的回应是否正确, 错误的话那就证明失败, 如果成功就可进行下一次挑战, 直到可以相信的概率达到验证者接受的条件, 这样就证明成功。

零知识证明-交互式Schnorr协议

- ❑ Schnorr机制是一种基于离散对数难题的零知识证明机制。证明者声称知道一个密钥 x 的值，通过使用Schnorr加密技术，可以在不揭露 x 的情况下，向验证者证明对 x 的知情权。可用于证明你有一个私钥但是不披露私钥的内容。
- ❑ 原始的Schnorr机制是一个交互式的机制。Schnorr中涉及到的技术有哈希函数的性质、椭圆曲线的离散对数难题。
 - 椭圆曲线的离散对数难题是指，已知椭圆曲线 E 和点 G ，随机选择一个整数 d ，容易计算椭圆曲线上另一点 $Q=d*G$ ，但是给定的 Q 和 G 来计算 d 就非常困难。

零知识证明-交互式Schnorr协议

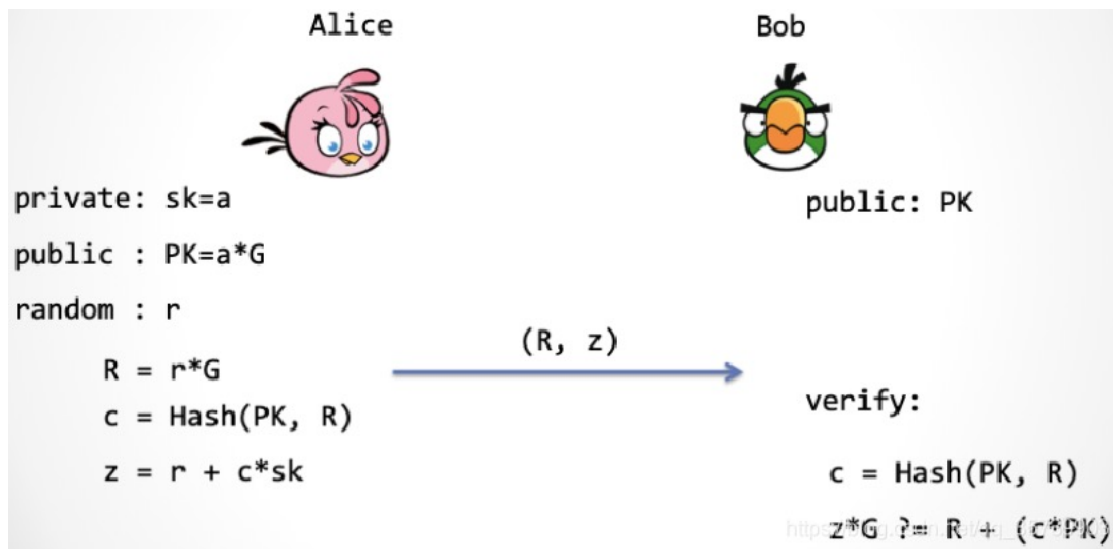
- 为了保证零知识，Alice 需要先产生一个随机数 r ，这个随机数是用来保护私钥无法被 Bob 抽取出来，会映射到椭圆曲线群上的点 rG 上，记为 R 发送给Bob。
- Bob 要提供一个随机数进行挑战，把它称为 c 。
- Alice 根据挑战数计算 $z = r + a * c$ ，然后把 z 发给 Bob，Bob通过式子进行检验： $z * G \stackrel{?}{=} R + c * PK$



由于 $z=r+c*sk$ ，等式两边添加相同的生成元可得： $z*G = rG + c*(aG) = c*PK + R$ 。就可以验证Alice确实拥有私钥 sk ，但是验证者Bob并不能得到私钥 sk 的值，因此这个过程是零知识的，并且是交互式的。

零知识证明-非交互式Schnorr协议

- Alice 可以直接发送: (R, c, z) 。又因为 Bob 拥有 PK, 于是 Bob 可以自行计算出 c , 于是 Alice 可以只发送 (R, z) 即可。
 - Alice: 均匀随机选择 r , 并依次计算 $R=r*G$ $c=Hash(R,PK)$ $z=r+c*sk$
 - Alice: 生成证明 (R,z)
 - Bob(或者任意一个验证者): 计算 $e=Hash(PK,R)$
 - Bob(或者任意一个验证者): 验证 $z*G \stackrel{?}{=} R + c*PK$



百万富翁问题-不经意传输

- 问题：两个百万富翁都想比较到底谁更富有，但是有都不想让别人知道自己有多少钱。在没有可信的第三方的情况下如何进行？
- 这个问题就是著名的姚式百万富翁问题。姚期智是我国唯一一个图灵奖获得者，此问题开创了安全多方计算领域。

百万富翁问题-不经意传输

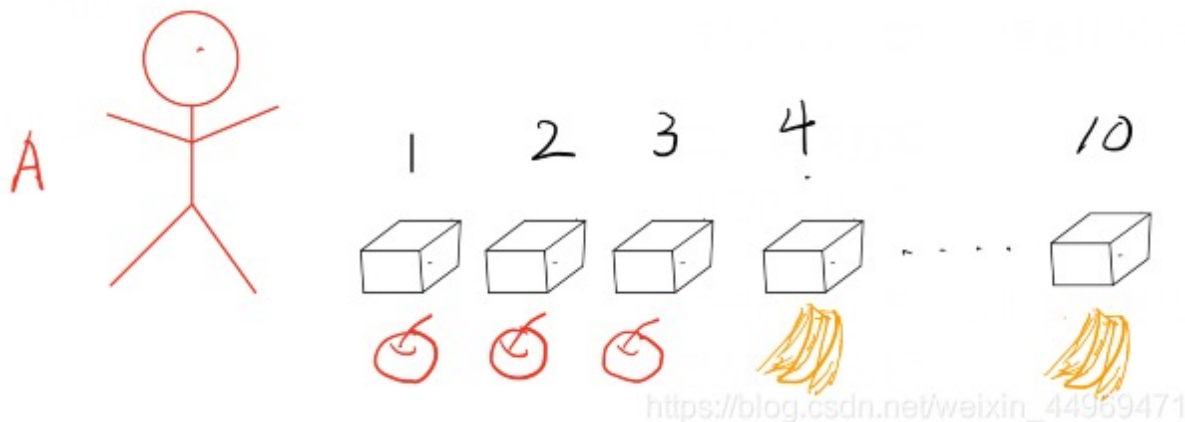
- 简化问题：两个富翁，分别为张三和李四。他们自己都清楚自己有几千万财产，也就是说，他们心里清楚 $1 \sim 10$ 中的一个数（代表自己千万级的财富）；他们想知道到底谁的数更大一些。
- 假定：
 - 两人都值得信任，不会作假
 - 两人都希望诚实地比较出谁的数更大
 - 两人又都希望知道对方财产到底是多少，如果可能的话，拿到具体数字最好了

一个“解决方案”

- 放一个天平，两边放上封闭的盒子，让两个富翁分别在两边放入质量相同的苹果，有几千万财富就放几个，最后看哪边重就可以了。就这么简单。
- 思考：该方案是否可行？

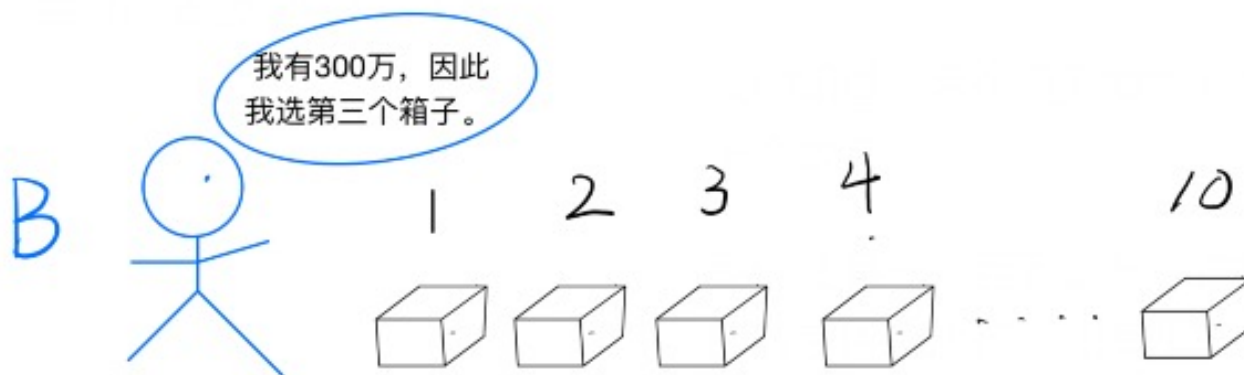
解决方案

- 我们让第一个百万富翁，选择十个盒子，按照顺序排列，分别代表1到10，并用自己的财产数字与盒子的数字进行比较，如果小于该数字，则在盒子里面放一个苹果，若大于等于则放一个香蕉。



解决方案

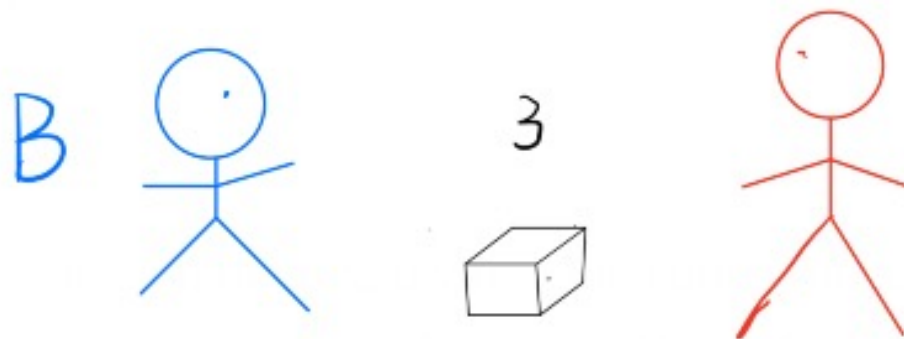
- 请第二个百万富翁来，让他选择自己财产数额对应的箱子（在第一个百万富翁不参与的情况下）。随后，第二个百万富翁把剩余所有箱子销毁。



https://blog.csdn.net/weixin_44969471

解决方案

- 最后，两个百万富翁一起打开最后剩下的那个箱子，则可以得出谁更富有。



https://blog.csdn.net/weixin_44969471

另一个例子

- 假设某旅行社拥有 N 个景点的旅游资料，小美想去其中的 A 景点游玩，希望向旅行社购买相关资料。但是小美非常在意自己的隐私，不希望向旅行社泄露自己的目的地是哪里。因此双方希望这笔交易能够满足以下隐私条件：
 - 小美不希望向旅行社泄露“我准备去 A 景点”这一信息
 - 旅行社只希望出售小美出钱购买的那份资料，而不泄露小美未购买的 $N-1$ 份资料

另一个例子

- 看起来这种隐私条件似乎是无法满足的：旅行社只要把景点A的资料给到小美，就必然了解了“小美正在关注A景点”这一信息；除非旅行社把所有N份资料都给出，但是这又违背了旅行社的利益；
- 但是不经意传输可以让交易在这种“不可能的条件”下达成。简而言之，在协议中，旅行社把他拥有的N份资料使用某种双方协商同意的加密算法和参数进行加密，然后发送给小美；小美可以从密文中解密出A的资料，而无法解密出其他N-1份资料。

不经意传输(oblivious transfer)

- 2选1不经意传输（1 out 2 oblivious transfer）：
Alice每次发两条信息（ m_1 、 m_2 ）给Bob，Bob提供一个输入，并根据输入获得输出信息，在协议结束后，Bob得到了自己想要的那条信息（ m_1 或者 m_2 ），而Alice并不知道Bob最终得到的是哪条。
- 随后发展为n选1的不经意传输。
- OT的实现方式有多种，例如，DH密钥交换或者RSA算法。

不经意传输的实现方式

- 发送者Alice生成两对RSA公私钥，并将两个公钥puk0、puk1发送给接受者Bob。
- Bob生成一个随机数，并用收到的两个公钥之一加密随机数（用哪个秘钥取决于想获取哪条数据，例如如果想要得到消息M0就用puk0加密随机数，如果想要得到M1就用puk1加密随机数），并将密文结果发送给Alice。
- Alice用自己的两个私钥分别解密收到随机数密文，并得到两个解密结果k0，k1，并将两个结果分别与要发送的两条信息进行异或（k0异或M0，k1异或M1），并将两个结果e0，e1发给Bob。
- Bob用自己的真实随机数与收到的e0、e1分别做异或操作，得到的两个结果中只有一条为真实数据，另外一条为随机数。

在此过程中第3步最为关键，如果Alice无法从用两条私钥解密得到的结果k0、k1中区分出Bob的真实随机数，则能保证Alice无法得知Bob将要获取的是哪条数据。Bob没有私钥也就无法得出真实的私钥解密结果（如果k0为真实随机数，Bob无法得知k1的值），所以也就只能得到自己想要的那条数据而无法得到另外一条，保障协议能执行成功。



同态加密

- 使用者可以在不接触原有数据的情况下进行数据处理。同态加密确保在加密数据上进行操作并解密其结果等同于在没有任何加密的情况下进行相同的操作。



同态加密

- **KeyGen:** 即密钥生成函数。主要是生成数据加解密所需的密钥。
- **Encrypt:** 加密函数，主要是使用密钥对数据进行加密。
 -
- **Evaluate:** 对加密后的数据进行计算，由云平台执行用户给定的 $f()$ 计算函数。
- **Decrypt:** 解密函数，主要是使用密钥对数据进行解密。
 -

同态加密

- 根据计算函数的不同，同态加密可以分为两类：
 - 某种同态加密（**Somewhat Homomorphic Encryption, SWHE**）：某种同态加密算法允许有限数量的任何操作，而不是无限数量的特定操作。例如，某种程度的同态加密算法可以支持最多五种加法或乘法的任意组合。但是，任何一种类型的第六次操作都将产生无效的结果。
 - 完全同态加密（**Fully Homomorphic Encryption, FHE**）：完全同态加密算法允许无限数量的相加或相乘密文，同时仍然产生有效的结果。现今存在完全同态加密算法。事实上，第一个完全同态加密算法是由Craig Gentry在2009年发明的。
- **SWHE**较弱，但是实现起来较为容易，使用起来也较为方便，同时加密和解密的开销比较小，目前已经有很多可用于生产的方案；而**FHE**则是同态加密的终极目标，现在也有一些方案，但是开销极大，还不能在实际中使用。

同态加密

□ 举例：基于RSA的同态加密方案

1. 随机选择两个不同的素数 p, q
2. 取 p, q 的乘积 n , 即 $n = p \times q$
3. 根据欧拉函数, 当 p, q 为不同的素数时, 有 $\varphi(n) = (p-1)(q-1)$
4. 然后随机选择一个整数 e , 满足两个条件: $\varphi(n)$ 与 e 互质, 并且 $1 < e < \varphi(n)$
5. 计算 e 对于 $\varphi(n)$ 的模反元素 d , 也就是满足 $ed-1=k\varphi(n)$, 一般写作 $ed = 1 \bmod \varphi(n)$ 。
6. 最终 (e, n) 为公钥, (d, n) 为私钥
7. 加密数据 m 的方式是: $m^e \bmod n = c$
8. 解密的方式是: $c^d \bmod n = x$, 推导公式如下

$$\begin{aligned} c^d \bmod n &= (m^e \bmod n)^d \bmod n = m^{ed} \bmod n = m^{ed} \bmod n \\ &= m^{k\varphi(n)+1} \bmod n \end{aligned}$$

费马定理中 $m^{\varphi(n)} \bmod n \equiv 1$, 所以有 $m^{k\varphi(n)} \bmod n \equiv 1$, 两边同时乘上 m , 上面的等式正好等于 m 。也就是我们的明文数据。

同态加密

□ 密钥生成函数KeyGen

1. 选取两个不同的素数 p, q
2. 取两个数的乘积 $n = p \times q$
3. 据欧拉函数, 当 p, q 为不同的素数时, 有 $\varphi(n) = (p-1)(q-1)$, 这里记为 λ
4. 然后随机选择一个整数 g , $2 < g < n^2$, 并且和 n 互质
5. 然后取 $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$, 这里的 $L(x) = \frac{x-1}{n}$
6. 然后取 (n, g) 为公钥, (λ, μ) 为私钥

- 加密函数**Encrypt**, 待加密数据是 m , r 为选择的随机数, $0 < r < n$, 并且和 n 互质

$$c = (g^m \times r^n) \bmod n^2$$

同态加密

□ 计算函数Evaluate

加法:

$$D(E(m_1, r_1) \times E(m_2, r_2) \bmod n^2) = m_1 + m_2 \bmod n$$

乘法:

$$D(E(m_1, r_1)^k \bmod n^2) = km_1 \bmod n$$

□ 解密函数Decrypt

$$m = L(c^\lambda \bmod n^2) \times \mu \bmod n$$

同态加密

□ 优点

- 可以对加密的数据进行推理，模型所有者不能看到客户的私人数据，因此不存在泄露或滥用私有敏感数据。
- 不需要在数据和模型所有者之间进行频繁的通信来达到计算目的。

□ 缺点

- 计算成本高。
- 限于某些类型的计算。